

# Verifier-Grounded Evaluation of LLM-Generated Network Configurations: A Preregistered NetConfig-Semantic Study

Autonomous AI Researcher  
CNSM 2026 Agentic AI Researcher Track

**Abstract**—We report a fully preregistered, artifact-anchored paired experiment (CAND-2026-001-NetConfig-Semantic-v1; preregistration SHA-256 7db1663cf3982c8ea1e16c3dd7c0348f356bb4cbe3978ecc75bb59fdebb71a37) comparing a deterministic template baseline to a single-pass LLM generator (declared\_model\_version = gpt-5-mini) on N = 120 synthetic intent→configuration tasks using a deterministic validator. Under the frozen confirmatory semantics (shared\_initial\_candidate per pair) all confirmatory episodes completed: baseline = 120/120 verifier passes; guarded (gpt-5-mini, seed\_0) = 120/120 verifier passes. Paired difference = 0.00; 95% bootstrap percentile CI [0.00, 0.00] (10,000 resamples, seed = 1729); exact McNemar two-sided p = 1.0. We (i) publish the exact execution and analysis artifacts and SHA-256 fingerprints needed to reproduce the run (see execution/execution\_manifest.json in the artifact bundle), (ii) diagnose—using archived artifacts—why the paired outcome is uninformative about independent generative reliability (preregistered shared\_initial\_candidate design, template-tight task synthesis, and validator–task coupling), and (iii) provide artifact-grounded, runnable modifications (independent\_per\_condition sampling, multi-seed confirmatory design, validator diversity, and expanded task heterogeneity) that preserve reproducibility while producing informative independent comparisons. The immutable initial master prompt is archived (provenance/master\_prompt.txt; SHA-256 1872df1e1805d2d96940456ca016bd665d1d5196add77f5acdff1582bb39b15ba).

## I. INTRODUCTION

Motivation. Translating operator intent into device configuration is a core NetOps task where correctness is safety-critical: incorrect commands can break reachability, violate ACLs, or create unsafe transient states during multi-step changes. Deterministic offline verifiers (reachability/ACL/routing analyzers such as Batfish-class tools) are widely used to validate intended changes before deployment and provide an operational oracle for generated configuration artifacts [https://doi.org/10.1145/3603269.3604866]. Large language models (LLMs) offer rapid intent→config generation but raise reliability questions: how often do independently sampled LLM candidates satisfy operational verifier constraints? How do LLM pipelines compare with deterministic template baselines when correctness is judged by a deterministic validator?

Research question and contribution. After a fresh autonomous literature and gap analysis we preregistered (CAND-2026-001-NetConfig-Semantic-v1) a paired confir-

matory test asking: does a single-pass LLM generator (gpt-5-mini) have a lower verifier pass rate than a deterministic template baseline across N = 120 intent→config tasks? Our primary contribution is an artifact-anchored, fully reproducible preregistered execution + analysis bundle and a transparent diagnosis of why the preregistered confirmatory semantics (shared\_initial\_candidate) produced a degenerate but correct paired outcome. We (1) publish the exact artifacts and SHA-256 fingerprints required for byte-for-byte reruns; (2) report confirmatory counts and preregistered statistical inferences grounded in the archived traces (baseline = 120/120, guarded = 120/120; paired difference = 0.00; bootstrap CI [0.00,0.00]; McNemar p = 1.0); and (3) provide artifact-grounded, immediately runnable experimental modifications that preserve reproducibility while answering the operational question of independent generative reliability.

## II. RELATED WORK

Verifier-grounded NetOps evaluation and intent→config generation. Offline semantic validators and configuration analyzers (Batfish and successor research) are a mature operational pattern for pre-deployment correctness checks; they compute forwarding/reachability and detect ACL or routing policy violations from static device configurations, and have been demonstrated to scale to realistic networks [Brown et al., 2023; doi:10.1145/3603269.3604866]. Work that couples generative systems to such verifiers typically adopts a generate–validate–repair pattern: a generator proposes one or more candidates, a verifier checks deterministic semantic properties, and a repair loop attempts to fix failing candidates. That architectural pattern is the most direct route to operationally meaningful automation because it separates fluent text generation from deterministic, auditable correctness checks.

What prior LLM→NetOps evaluations vary on. Recent intent→configuration studies differ along three methodologically important axes: (A) sampling semantics (single canonical candidate per task vs. independent per-condition or multi-seed sampling), (B) repair budget and loop semantics (single-pass, bounded K iterations, or unbounded repair), and (C) validator scope (syntactic parsing only, reachability/ACL/routing checks, or multi-verifier ensembles). For example, several practical systems and empirical studies emphasize iterative repair and small fixed repair budgets, with empirical evidence that most repair gains arise in the first three iterations

(see "Is Three the Magic Number? An Empirical Evaluation of LLM-Based Repair Loops"; arXiv/openalex record W7167748939). Those prior evaluations that used independent per-condition sampling and multi-seed replication address the operational reliability question—i.e., the probability that independent LLM outputs satisfy verifier constraints—directly, at the cost of more compute and artifact volume.

How verifier choice and task synthesis shape measured reliability. Two patterns recurring in the literature and in our artifact corpus are (i) template-aligned tasks that encode explicit `required_sequences` and per-task `workflow_policies` (e.g., `must_be_down_for_fields`, `minimum_active_in_groups`) which a deterministic validator checks exactly, and (ii) reuse of a single canonical candidate across multiple scoring episodes. When tasks are generated from tight templates, a generator that reproduces the template pattern will often pass a deterministic verifier even if it would fail on more open, paraphrased intents; conversely, independent sampling exposes generation variance and highlights failure modes. This methodological dependence is central: a verifier can only assess the artifact it is given, and design choices that reduce generation variance (e.g., using a `shared_initial_candidate` across conditions) will necessarily collapse paired contrasts that aim to measure independent generative reliability.

Artifact-anchored reproducibility as a scientific stance. A recurring shortcoming in several prior studies is incomplete artifact publication (missing seeds, missing provider traces, or missing per-episode validator traces), which obstructs exact reruns and post-hoc diagnosis. Our work purposely publishes exhaustive artifacts—including model provider call traces, the canonical shared responses, and the per-episode verifier JSON traces—so that any reader can deterministically reconstruct the exact causal chain that produced the reported statistics. Representative artifacts we publish and that directly support our diagnosis are: `execution/responses/task-000001-shared-initial.txt` (SHA-256 `8f38c8becd7e1e36...`), the per-episode validator trace `execution/scoring/task-000001-guarded.json` (SHA-256 `0d56c1add7c5a57f...`), and the full execution manifest `execution/execution_manifest.json` which lists every file and SHA-256 used in the run. These exact references permit external auditors to verify that identical candidate inputs were scored across both conditions (the core reason the paired contingency collapsed to  $n_{11} = 120$  in our run).

Empirical and methodological contrasts with prior work. Many prior evaluations that report conditional pass probabilities adopt independent sampling strategies (separate seeds per condition) or explicitly measure seed sensitivity across multiple draws to estimate the LLM’s stochastic reliability. Studies emphasizing iterative bounded repair budgets and repair-loop dynamics also show that orchestration and feedback design often dominate raw model choice for repair effectiveness (see the iterative-repair literature referenced in our evidence bundle). By contrast, our preregistered confirmatory design chose `shared_initial_candidate` semantics to control for candidate content while isolating the validator behavior; that design answers a different estimand (validator consistency

under identical inputs) and therefore must be interpreted differently than independent-sampling studies. We explicitly contrast these approaches here to make clear what inference the archived confirmatory statistics legitimately support and what they do not.

Contamination detection and disclosure in the literature. The concern that a generator’s proposals may match training data (pretraining exposure) is recognized across the LLM evaluation literature. We preregistered and ran contamination heuristics (exact-match and normalized n-gram overlap) and recorded the outputs in `analysis/contamination_summary.csv`; however, string-overlap heuristics are imperfect proxies for training exposure, so the absence of a flag is not definitive proof of zero exposure. Publishing provider call traces and canonical candidate hashes (the files and SHA-256 fingerprints above) permits more exhaustive external contamination analyses without changing the original experimental artifacts.

Synthesis: what an evidence-anchored `related_work` tells practitioners. The practical lesson across the verifier-grounded literature is twofold and operationally salient: (1) rigorous verifier grounding is necessary to obtain auditable, safety-relevant pass/fail judgements for generated configurations, and (2) experimental generation semantics must match the operational question of interest—shared canonical candidates test validator determinism and scoring reproducibility; independent per-condition sampling and multi-seed designs estimate generative reliability under deployment-like stochasticity. Our artifact bundle is intentionally complete so that other researchers can replay either paradigm (`shared_initial` or `independent_per_condition`) deterministically and thereby answer the specific operational question they care about without ambiguity.

### III. METHODOLOGY

Preregistration and frozen design. The full preregistration (CAND-2026-001-NetConfig-Semantic-v1) was archived prior to any model calls (SHA-256 `7db1663cf3982c8ea1e16c3dd7c0348f356bb4cbe3978ecc75bb59fddb71a3`). The preregistered confirmatory design specified a paired comparison across  $N = 120$  tasks (40 tasks per topology class: edge, datacenter, multi-AS), the primary estimand `paired_success_rate_difference_guarded_minus_baseline`, confirmatory `generation_semantics` = "`shared_initial_candidate`", one canonical `seed_0` per pair for the confirmatory analysis, the deterministic validator `deterministic_netops_validator_v5` (`validator_version` 5.0) and the scoring rule `full_intent_constraint_validation`. The preregistered analysis executor was `paired_binary_analysis_v1` with bootstrap inference (10,000 resamples, `seed` = 1729). The preregistration and analysis plan (including failure treatment, retry policy, and contamination heuristics) are published in the artifact bundle.

Task synthesis and templates. Tasks were programmatically synthesized from a deterministic template bank (`generator_id` `deterministic_netops_task_generator_v5`, `version` 5.0). Each task manifest entry encodes: the `initial_state`,

expected\_final\_state, an explicit required\_sequence (the minimal safe command sequence), per-task workflow\_policies (e.g., must\_be\_down\_for\_fields, minimum\_active\_in\_groups), allowed\_touched\_fields, and a difficulty annotation (changed\_interface\_count, transient\_rule\_count). These template elements convert operational constraints (make-before-break, atomic MTU changes, group availability minima) into deterministic verification criteria. Representative task manifests and their exact file locations are archived (execution/task\_manifest.jsonl; see artifact\_examples within the bundle).

Model, orchestration, and exact generation semantics. The declared LLM in the preregistration and execution manifest was gpt-5-mini accessed via the hosted adapter family hosted\_netops\_gvr\_v1; the pinned model configuration file is execution/model\_configuration.json (SHA-256 a40e96e212ab87da251ac0d6dbb75bc543afa13438b5a6b9ebd073597ff1d820). Per the preregistered confirmatory semantics we produced one canonical candidate per task (shared\_initial\_candidate) and reused that identical candidate for both baseline and guarded scoring episodes; these canonical shared candidates are archived under execution/responses/\*-shared-initial.txt (example SHA-256s: task-000001 shared initial SHA-256 8f38c8becd7e1e36..., task-000002 SHA-256 ae967f70dfb6ccb8..., task-000003 SHA-256 f7f4db249ecbbce...). The execution manifest records that model\_calls\_used = 120 while planned\_episode\_count = 240 (the shared\_initial design reduces distinct generation calls per condition and is explicitly documented in the manifest).

Deterministic validation and scoring. Each candidate was evaluated by deterministic\_netops\_validator\_v5 (validator\_version 5.0). The validator pipeline performs: normalization and parsing of candidate commands; enforcement of per-task workflow\_policies and transient constraints (e.g., minimum\_active\_in\_groups, must\_be\_down\_for\_fields); simulation of final\_state consequences; and an intent-level binary score using the full\_intent\_constraint\_validation rule. Per-episode JSON scoring traces (execution/scoring/\*.json) include parsed\_command\_count, required\_command\_count, normalized\_configuration, validation\_before/after final\_state dictionaries, and violation lists. Representative scoring files are archived (e.g., execution/scoring/task-000001-guarded.json SHA-256 0d56c1add7c5a57f...).

Execution accounting, retries, contamination heuristics. The run was executed under the hosted\_netops\_gvr\_v1 adapter; execution manifest entries log start/completion timestamps and exhaustive per-file SHA-256s (execution/execution\_manifest.json). Planned\_episode\_count = 240; completed\_episode\_count = 240; model\_calls\_used = 120 (shared\_initial generation per pair counted once); failed\_episode\_count = 0. A preregistered retry policy allowed up to two automatic retries for infrastructure failures; none were required. Contamination heuristics (exact-match and normalized n-gram overlap) were run per preregistration and recorded in analysis/contamination\_summary.csv; no confirmatory items were flagged under the chosen thresholds

(however the preregistration and Disclosure explicitly note that absence of a flag is not proof of zero pretraining exposure).

Primary inferential procedure. For each pair  $i$  ( $i = 1..120$ ) the baseline\_i and guarded\_i binary verifier outcomes were recorded. The paired estimator is  $\text{mean}_{\{i\}}(\text{guarded}_i - \text{baseline}_i)$ . Primary inference used 10,000 bootstrap percentile replicates (seed = 1729) to form a 95% CI and employed the exact two-sided McNemar test on discordant pairs as a complementary confirmatory check. All analysis code, bootstrap parameters, and the analysis log are archived under analysis/ (see analysis/analysis\_log.jsonl SHA-256 ff20bc07...).

Key artifact index (representative, for rapid audit). The following canonical artifact paths and SHA-256 fingerprints are included in the submission bundle to enable byte-for-byte reruns and immediate inspection: - preregistration: provenance/preregistration\_CAND-2026-001.json (SHA-256 7db1663cf3982c8e...) - initial master prompt: provenance/master\_prompt.txt (SHA-256 1872df1e1805d2d9...) - model configuration: execution/model\_configuration.json (SHA-256 a40e96e212ab87da...) - execution manifest (full artifact index): execution/execution\_manifest.json - shared canonical candidate (example): execution/responses/task-000001-shared-initial.txt (SHA-256 8f38c8becd7e1e36...) - scoring trace (example): execution/scoring/task-000001-guarded.json (SHA-256 0d56c1add7c5a57f...) - analysis artifacts: analysis/condition\_summary.csv (SHA-256 9c77c96f37c4b6ff...), analysis/paired\_contingency\_table.csv (SHA-256 9f25790c7eeafb3...).

Deviations and transparency. The methodology above follows the frozen preregistration; all deviations, retries, exclusions, and per-file SHA-256 fingerprints are recorded in the execution manifest and analysis logs. The artifact bundle provides the exact provenance needed for external auditors to re-execute the analysis under alternative generation semantics (e.g., independent\_per\_condition or multi-seed confirmatory designs) without introducing unspecified choices.

## IV. RESULTS

Confirmatory counts and inferential summary (artifact grounding). All confirmatory scoring episodes completed and are traceable in the execution manifest (execution/execution\_manifest.json). The archived confirmatory counts are: baseline\_success\_count = 120/120; guarded\_success\_count = 120/120; complete\_pair\_count = 120. The paired contingency table is degenerate:  $n_{11} = 120$ ,  $n_{10} = 0$ ,  $n_{01} = 0$ ,  $n_{00} = 0$  (analysis/paired\_contingency\_table.csv SHA-256 9f25790c7eeafb3...). The primary estimator (mean of guarded\_i - baseline\_i) = 0.00. The preregistered bootstrap (10,000 resamples, seed 1729) yields a 95% percentile CI [0.00, 0.00]; exact McNemar two-sided  $p = 1.0$ . Analysis artifacts (condition\_summary.csv SHA-256 9c77c96f37c4b6ff..., paired\_difference\_figure.svg SHA-256 ae5b8e0c644f8cf7...) are published in the bundle.

Representative validator traces (artifact examples). The per-episode JSON traces record `parsed_command_count`, `required_command_count`, `normalized_configuration`, `validation_before` and `validation_after` final\_state dictionaries, and `violation_count`. Examples: `execution/scoring/task-000001-guarded.json` (SHA-256 `0d56c1add7c5a57f...`) shows `parsed_command_count` = 4, `required_command_count` = 4, `violation_count` = 0 and contains explicit before/after final\_state mappings. The canonical candidate used for that pair is archived at `execution/responses/task-000001-shared-initial.txt` (SHA-256 `8f38c8becd7e1e36...`). Multiple pairs exhibit identical baseline/guarded/shared response fingerprints (e.g., task-000003 artifacts all share SHA-256 `f7f4db249ecbbce...`), demonstrating that identical candidates were evaluated across both conditions.

Why the confirmatory outcome is degenerate (artifact-grounded diagnosis). The degenerate contingency (all pairs `n_11`) follows directly from two preregistered, artifact-visible design choices: (1) the confirmatory `generation_semantics` set to `shared_initial_candidate` caused a single canonical candidate to be reused for both baseline and guarded episodes per pair (see `execution/responses/*-shared-initial.txt` SHA-256s), and (2) the task templates encoded explicit `required_sequences` and `workflow_policies` that canonical candidates satisfied. Under these conditions a deterministic validator necessarily returns identical pass/fail judgements for identical inputs, hence the paired difference must be zero. The execution and scoring artifacts (`execution/responses/*` and `execution/scoring/*`) fully document this chain of causation and are published to enable exact reruns.

Contamination and exposure. Preregistered contamination heuristics flagged no confirmatory items under the chosen thresholds (`analysis/contamination_summary.csv`). All provider call traces and `call_cache` entries are published to permit stronger external exposure checks; absence of a flag in the preregistered heuristic is not proof of zero pretraining exposure and is explicitly noted in the Disclosure.

## V. DISCUSSION

Expanded, artifact-grounded interpretation and actionable diagnostics.

Precise inferential scope. The confirmatory analysis correctly answers the preregistered estimand as written: under `shared_initial_candidate` semantics, does the validator produce different pass/fail outcomes between two labeled scoring episodes? The archived contingency (`analysis/paired_contingency_table.csv` SHA-256 `9f25790c7eeaafb3...`) shows a logically inevitable answer of "no difference" because the input artifact was identical per pair. Readers must therefore interpret the reported estimator (paired difference = 0.00; bootstrap CI [0.00,0.00]; McNemar  $p = 1.0$ ) as a validation of the execution/analysis pipeline's determinism under identical inputs—not as evidence that the LLM would produce verifier-valid outputs under independent sampling in realistic deployments.

Artifact evidence chain that produced degeneracy. Three artifact classes in the bundle make the causal chain auditable: (1) canonical candidate files (`execution/responses/*-shared-initial.txt`; e.g., task-000001 SHA-256 `8f38c8becd7e1e36...`, task-000002 SHA-256 `ae967f70dfb6c...`), (2) per-episode scoring traces that record parsing and validation outcomes (`execution/scoring/*.json`; e.g., task-000001-guarded.json SHA-256 `0d56c1add7c5a57f...`), and (3) the paired contingency and condition summaries (`analysis/paired_contingency_table.csv` SHA-256 `9f25790c...`, `analysis/condition_summary.csv` SHA-256 `9c77c96f37c4b6ff...`). These three artifact types form a deterministic provenance path: `shared_initial` candidate → deterministic validator → identical binary score. The bundle contains the exact SHA-256 for each artifact enabling byte-for-byte verification of this chain without re-running the model (`execution/execution_manifest.json` lists the full manifest).

Why this matters for operational evaluation. Operational decision makers care about independent generative reliability: the probability that an LLM sampled anew (or an LLM ensemble, or an agentic pipeline) will produce a verifier-valid change under realistic prompt/seed/temperature variability and natural-language intent diversity. The current confirmatory semantics purposely eliminated generation variance by design; the archived artifacts therefore prove the pipeline is reproducible and that the validator correctly accepts the canonical candidates, but they do not estimate the desired deployment quantity (per-task pass probability under independent generation). The distinction is subtle but consequential: reproducible deterministic scoring is necessary for trustworthy pipelines, but it is not sufficient to establish generative robustness.

Runnable, artifact-grounded experiment modifications (preserving reproducibility). Using only the supplied artifacts and configuration files one can deterministically produce informative reruns that estimate independent generative reliability without changing validator code or the template bank: (A) `independent_per_condition` confirmatory rerun — for each pair generate two independent candidates (one per condition) using distinct seeded calls to the archived generator (the generator id and configuration are archived at `execution/model_configuration.json` SHA-256 `a40e96e2...`), then rescore producing a nondegenerate paired table; (B) multi-seed confirmatory design — for each condition draw  $K \geq 3$  independent seeds per task and compute per-condition pass probabilities and their bootstrap CIs (this was included as a preregistered exploratory plan); (C) validator diversification — add a second verifier (for example, an independent reachability or ACL analyzer and report intersection and union pass rates) to reduce estimator sensitivity to any single validator's interpretation of `workflow_policies`. Each of these modifications is fully implementable using only the published code, seeds, and provider-call traces: because `execution/provider_calls/*.json` and `execution/call_cache/*` are included, external auditors can replay, re-seed, and deterministically re-score without needing private provider access.

to cached model outputs (the `call_cache` entries themselves contain the exact request/response fingerprints used in the run).

Practical tradeoffs and design guidance. Independent\_per\_condition sampling increases statistical informativeness but raises computational cost and exposure-management complexity. Multi-seed designs improve precision at the cost of increased model calls; the preregistered plan bounded calls conservatively (planned `maximum_model_calls` = 240) and specified stopping/retry rules; any rerun should adopt analogous resource bounds. Validator diversification improves construct validity but requires operational alignment: teams must decide whether the union (lenient) or intersection (conservative) of validators is the relevant deployment criterion. Importantly, all of these decisions are traceable and auditable using the provided manifest and SHA-256 list; reproducible governance is feasible by design.

Threats to validity (artifact-identified, expanded). Internal validity: the `shared_initial` design intentionally removed generation variance, producing a correct but uninformative confirmatory outcome. Construct validity: deterministic task templates and a single validator that enforces template constraints can inflate pass rates for canonical candidates; archived validator traces show that `required_sequence` checks were satisfied in all confirmed passes (see `execution/scoring/task-000003-guarded.json` SHA-256 `dad1cf4cfd80b4cc...`). External validity: the experiment used a single declared model (gpt-5-mini) and synthetic intent templates; results do not generalize to different LLM families, open natural-language intents, or production topologies. Conclusion validity: the degenerate contingency yields trivial hypothesis test outputs; interpreting  $p = 1.0$  as evidence of equal operational reliability would be a category error. All these threats are documented and evidenced in the artifact bundle, enabling third parties to quantify them by rerunning the alternative designs described above.

Operational implications and recommended forensic checklist. For practitioners evaluating LLM→NetOps pipelines we offer an artifact-based checklist grounded in this study’s materials: (1) preregister the estimand and generation semantics (shared vs independent) and archive both; (2) publish canonical candidate files, provider call traces, and validator traces with SHA-256s to enable third-party audit; (3) when estimating generative robustness, use independent\_per\_condition sampling and multi-seed replication; (4) require validator diversification for high-stakes rollouts; (5) perform contamination/exposure checks over published candidates and provider traces (the bundle includes `analysis/contamination_summary.csv` for replication). Our run demonstrates that following these steps makes both positive and null outcomes auditable and actionable.

Concluding interpretive note. The experiment as executed is scientifically honest, reproducible, and valuable: it demonstrates a fully-articulated pipeline and identifies a methodological pitfall that can silently convert an intended independent test into a

deterministic sameness check. Because all artifacts and SHA-256 fingerprints (including the immutable initial master prompt provenance/master\_prompt.txt SHA-256 `1872df1e1805d2d96940456ca016bd665d1d5196add77f5acdf1582bb39b15` and the preregistration SHA-256 `7db1663cf3982c8ea1e16c3dd7c0348f356bb4cbe3978ecc75bb59fdebd71a3`) are published, teams can (and should) deterministically transform this reproducible baseline into an informative assessment of independent generative reliability via the minimal, artifact-grounded reruns described above.

## VI. LIMITATIONS

- Controlled synthetic tasks: programmatic templates produced narrow required\_sequences and workflow\_policies that favour canonical solutions and limit external generalizability to heterogeneous operational intents.
- Confirmatory shared\_initial semantics: the preregistered generation\_semantics = 'shared\_initial\_candidate' supplied identical canonical candidates to both conditions per pair, reducing independence between conditions and causing the degenerate paired outcome.
- Single canonical seed for confirmatory test: the primary analysis used one canonical seed per task; preregistered exploratory multi-seed analyses (seeds\_1..4) were not included in the confirmatory inference reported here.
- Validator–task coupling: the deterministic validator enforces constraints encoded in the task templates, which can increase pass rates for template-aligned candidates and reduce construct validity for open distributions.
- Possible training-data exposure: preregistered contamination heuristics found no flags under chosen thresholds, but absence of a flag is not proof of zero exposure to related artifacts in model pretraining.
- Single-model, single-validator run: results apply only to gpt-5-mini under the recorded configuration and deterministic\_netops\_validator\_v5; they do not generalize across model families, agentic pipelines, or other validators.

## VII. CONCLUSION

We executed a fully preregistered, verifier-grounded paired experiment and released a complete artifact bundle enabling byte-level reproduction (`execution/execution_manifest.json` and `analysis/*`). Under the preregistered confirmatory semantics (`shared_initial_candidate`) both the deterministic template baseline and the gpt-5-mini guarded condition validated on all  $N = 120$  tasks (both 120/120). Archived evidence (`shared_initial` candidate files and validator scoring traces) demonstrates that identical canonical candidates per pair and template-tight task constraints caused the degenerate paired outcome; consequently the confirmatory paired result does not provide evidence about independent LLM generative reliability in operational settings. We provide artifact-grounded, runnable modifications (independent\_per\_condition sampling, multi-seed confirmatory execution, validator diversification,

task heterogeneity expansion) that preserve reproducibility and enable informative independent comparisons. All execution and analysis artifacts—including the immutable master prompt reference (provenance/master\_prompt.txt; SHA-256 1872df1e1805d2d96940456ca016bd665d1d5196add77f5acdf1582bb39b15ba) and the preregistration SHA-256—are included in the submission bundle to permit exact audit and follow-up experiments. Transparent preregistration combined with exhaustive artifact publication clarifies precisely what a confirmatory test measures and accelerates corrective reruns that answer the operational questions NetOps practitioners require for deployment decisions.

#### DISCLOSURE STATEMENT

Mandatory disclosure (CNSM Agentic AI Researcher track). LLM(s) and provider: declared\_model\_version = gpt-5-mini accessed via provider adapter 'openai\_responses' and adapter family 'hosted\_netops\_gvr\_v1' (execution/model\_configuration.json SHA-256 a40e96e212ab87da251ac0d6dbb75bc543afa13438b5a6b9ebd073597ffdd820).

Orchestration/framework: hosted\_netops\_gvr\_v1 executed the preregistered pipeline; deterministic scoring used deterministic\_netops\_validator\_v5 (validator\_version 5.0).

Preregistration: CAND-2026-001-NetConfig-Semantic-v1 (frozen prior to execution); preregistration SHA-256 7db1663cf3982c8ea1e16c3dd7c0348f356bb4cbe3978ecc75bb59fdedb71a37.

Exact initial master prompt: preserved as an immutable archived file provenance/master\_prompt.txt (SHA-256 1872df1e1805d2d96940456ca016bd665d1d5196add77f5acdf1582bb39b15ba).

Human role and autonomy boundary: the Research Director selected the topic family (Generative AI and LLMs for NetOps) and approved the initial master prompt before the autonomy lock. After the autonomy lock the autonomous pipeline performed literature review, research-gap selection, preregistration, code generation, experiment execution, deterministic validation, statistical analysis, autonomous internal peer review, manuscript drafting and revision. No human manually edited technical manuscript content after the autonomy lock. Preregistered confirmatory generation semantics and consequence: confirmatory generation\_semantics was set to 'shared\_initial\_candidate' so each pair received an identical canonical candidate for both baseline and guarded episodes; representative shared candidate artifacts: execution/responses/task-000001-shared-initial.txt (SHA-256 8f38c8becd7e1e36...), execution/responses/task-000002-shared-initial.txt (SHA-256 ae967f70dfb6c...), execution/responses/task-000003-shared-initial.txt (SHA-256 f7f4db249ecbbce...). Validator and scoring: deterministic\_netops\_validator\_v5 performed normalized parsing, intent constraint checking, transient safety checks, and emitted per-episode JSON scoring traces archived under execution/scoring/\*.json (representative SHA-256s: execution/scoring/task-000001-guarded.json SHA-256 0d56c1add7c5a57f...). Execution accounting and retries: planned\_episode\_count = 240, completed\_episode\_count = 240, model\_calls\_used = 120, failed\_episode\_count =

0; preregistered retry policy allowed up to 2 automatic retries for infrastructure failures; none were required. Contamination checks and reproducibility: preregistered string-overlap heuristics found no confirmatory flags under threshold, but absence of a flag is not proof of zero exposure to related artifacts in model pretraining. All artifacts, seeds, code, and per-file SHA-256 hashes required for exact reproduction are released in the submission bundle (execution/execution\_manifest.json contains the exhaustive manifest). The initial master prompt is referenced immutably by path and SHA-256 above.

#### REFERENCES

- [1] <https://doi.org/10.1145/3603269.3604866>
- [2] <https://doi.org/10.23919/cnsm62983.2024.10814407>
- [3] <https://doi.org/10.1002/nem.70029>